

CV GTSRB Evaluation

Moritz Anton Zideck

16th May 2026

Disclaimer Regarding Missing Results

I am very sorry that the final plots, CSV files, and detailed result tables are not included in this current version of the documentation.

The experiments were run on a remote PC, where all generated result files, plots, and CSV exports are stored. Unfortunately, I temporarily lost access to this remote machine during the weekend and will only be able to access the data again on Monday.

Because of this, the current documentation is based on the experiment setup, methodology, and the results I can remember from the training process. The exact numerical results, full plots, CSV files, and final detailed analysis will be added as soon as I regain access to the remote PC.

I apologize for the inconvenience and will update the documentation with the complete result data as soon as possible.

1 Introduction

The goal of this project was to test different image processing strategies and data augmentation methods on the GTSRB Traffic Sign Recognition Benchmark dataset.

GTSRB is a traffic sign classification dataset. The task is to classify each input image into the correct traffic sign class. The dataset contains 43 traffic sign classes, 39,209 training images, and 12,630 test images.

The main question of this project was:

Can image processing and augmentation strategies improve the classification performance of deep learning models on the GTSRB dataset?

To speed up the workflow, I used my own computer vision pipeline library called Vision Studio:

https://github.com/mozi30/CV_VisionStudio

Vision Studio was used to make the training, inference, preprocessing, and experiment workflow easier and faster.

2 Dataset

The dataset used in this project was the German Traffic Sign Recognition Benchmark, also called GTSRB.

It is a multi-class image classification dataset with German traffic signs. Each image belongs to one of 43 classes. The dataset is useful for testing computer vision models because traffic sign images can have different lighting conditions, image qualities, positions, and visual appearances.

During the project, it also became clear that the classes are not equally represented. Some traffic sign classes have more samples than others. Because of this class imbalance, additional training strategies such as weighted loss and balanced sampling were tested.

3 Project Tools

For this project, I used my own computer vision pipeline library:

Vision Studio

https://github.com/mozi30/CV_VisionStudio

The library was used to speed up the process of:

- loading and preparing the dataset
- applying image processing strategies
- applying augmentations
- training models
- running inference
- comparing experiments

This made it easier to test multiple strategies in a structured way.

4 Hardware

All models were trained on an RTX 5070.

The experiments were run on a remote PC. At the time of writing this documentation, I currently do not have access to the remote PC because access was lost during the weekend. Because of this, the final plots, CSV files, and full result analysis will be added later when access is available again.

5 Base Preprocessing

For the baseline and for all other tested strategies, the input images were prepared in the same basic way:

- images were resized to 224×224
- pixel values were normalized to the range 0–1

This created a consistent input format for all models.

6 Tested Image Processing Strategies

Several image processing strategies were tested to see whether they could improve classification performance.

6.1 Baseline

The baseline used only the base preprocessing:

- resize to 224×224
- normalize pixel values to 0–1

This baseline was used as the reference point for comparing all other strategies.

6.2 Histogram Equalization

Histogram equalization was tested to improve the contrast distribution of the images.

This strategy performed best among the image processing methods. It was the only image processing method that clearly improved the performance compared to the baseline.

A possible reason is that histogram equalization works like a type of image normalization. It can make lighting and contrast more consistent between images.

6.3 Contrast and Brightness Normalization

Contrast and brightness normalization was tested to reduce the influence of different lighting conditions.

The goal was to make images more visually consistent, especially because traffic signs can appear under different illumination conditions.

6.4 Sobel Filter

A Sobel filter was tested to extract edge information from the image.

However, this strategy performed the worst. The likely reason is that Sobel filtering removes too much fine image information. For traffic sign classification, color, texture, symbols, and small details are important. If too much information is removed, the model has less useful input for classification.

6.5 Custom Traffic Sign Color Boost

A custom traffic sign color boost was tested.

The idea was to take the special color ranges that are common in German traffic signs and boost them. This should make important traffic sign colors more visible to the model.

6.6 Edge Sharpener for Large Structures

An edge sharpening method was also tested.

The goal was to boost larger edges and structures of traffic signs inside the image. This was intended to make the traffic sign shape more visible and easier to detect for the model.

7 Data Augmentation

In addition to image processing, data augmentation was tested during training.

The following augmentations were used:

- random resized crop
- random rotation between -5° and $+5^\circ$
- random blur
- random color changes
- random brightness changes

The goal of augmentation was to create more variation during training. This can help the model generalize better because it sees slightly different versions of the same training images.

The improvement from augmentation was visible, but not very large. A likely reason is that the GTSRB dataset is already quite large and already contains different lighting conditions and visual variations. Because of this, augmentation helped, but it was not the biggest factor.

8 Class Imbalance Handling

Later in the training process, it became clear that not all classes were equally represented in the dataset.

Because of this, two methods were tested:

8.1 Weighted Loss

Weighted loss was used based on class probability. Classes with fewer samples received higher importance during training.

This helped the model pay more attention to underrepresented classes.

8.2 Balanced Sampling

Balanced sampling was also tested. The goal was to sample the training data in a way that reduces the effect of class imbalance.

Both weighted loss and balanced sampling showed an improvement compared to the baseline, especially with the custom model.

9 Models

Three model types were tested.

9.1 ResNet-50

ResNet-50 was one of the main models used in the project.

It achieved the best general performance overall. Based on the results, ResNet-50 reached around 97%–99% accuracy in different experiment setups.

Because of its strong performance, ResNet-50 was later used for the final best-model experiments.

9.2 Swin Transformer-T

Swin Transformer-T was also tested.

This model already became stable and achieved good performance after the base training. Because of that, it did not require the same amount of additional optimization as the other models.

The performance is around 97%–98% accuracy and 95%–97% F1 score.

9.3 Custom Hybrid Model

A custom model was also created.

The idea behind this model was to combine:

- convolution layers for local image analysis
- a low-level transformer part for a more global understanding of the image

The convolution part was used to analyze local visual patterns, while the transformer part was intended to help understand the overall structure of the image and support traffic sign detection and classification.

This created a hybrid approach between CNN-based local feature extraction and transformer-based global reasoning.

The custom model reached around 95%–97% accuracy. While this was lower than ResNet-50 and Swin Transformer-T, it was still a strong result because the custom model was smaller and faster.

10 Training Pipeline

The models were first trained with the Adam optimizer to get a stable base training.

After the base training, additional fine-tuning was done using both:

- Adam
- SGD

The goal was to compare which optimizer worked better for optimization after the first training stage.

In the experiments, SGD fine-tuning generally performed better than Adam fine-tuning. Because of this, SGD was used in the later optimization steps for the best model experiments.

The only exception was Swin Transformer-T, which already reached stable and good performance after base training.

11 Results Summary

The full result analysis, plots, and CSV files are currently not available because the experiments were run on a remote PC that I temporarily cannot access.

Because of that, this section currently describes the results from memory. The final numbers, plots, and CSV-based analysis will be added later.

11.1 General Model Performance

All tested models performed well on the GTSRB dataset.

11.2 Image Processing Results

The image processing strategies often made the performance worse instead of better.

The likely reason is that some processing methods removed useful information from the image or created unnatural inputs. Modern deep learning models can often learn useful features directly from the original image, so strong image processing can sometimes hurt instead of help.

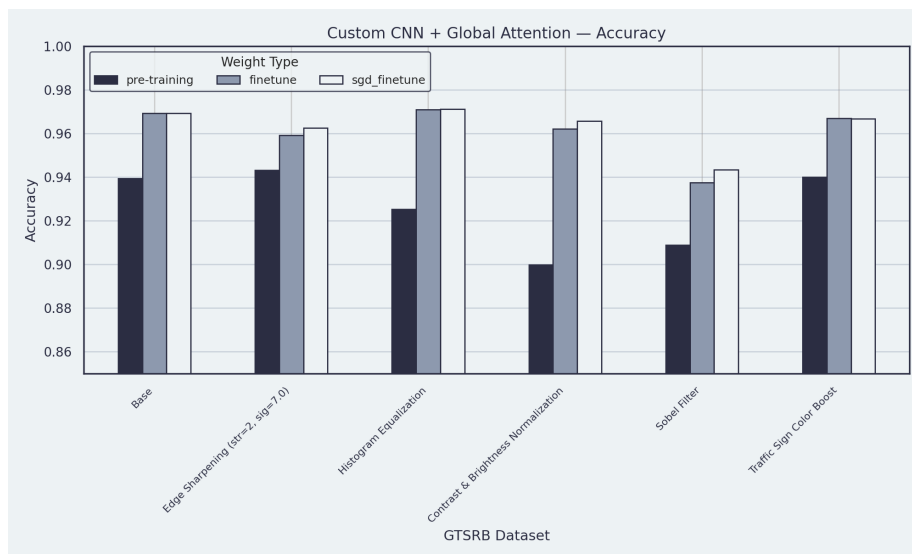


Figure 1: Custom Model Accuracy

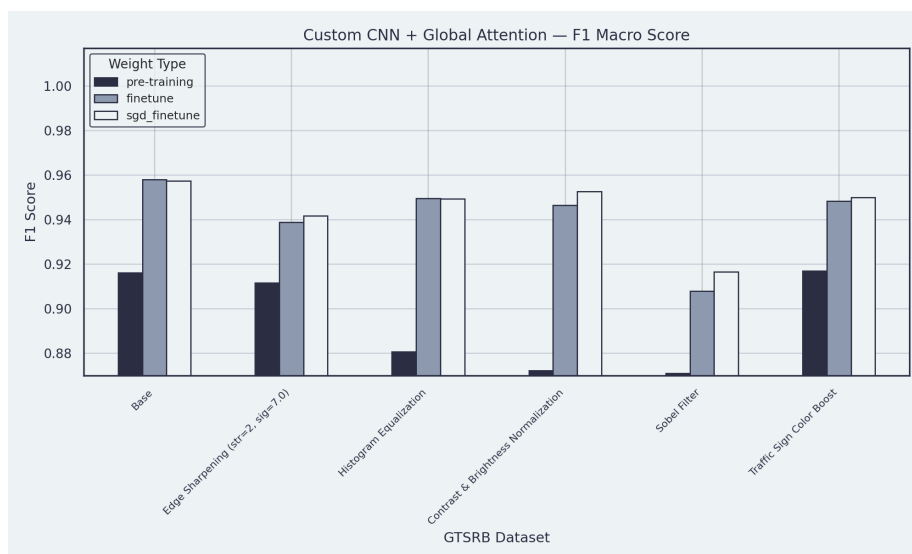


Figure 2: Custom Model F1 Macro Score

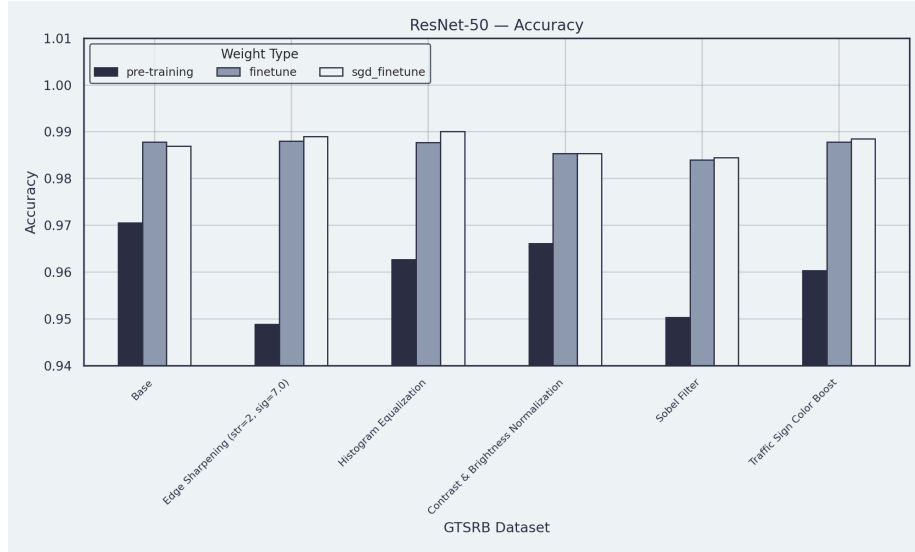


Figure 3: ResNet50 Accuracy

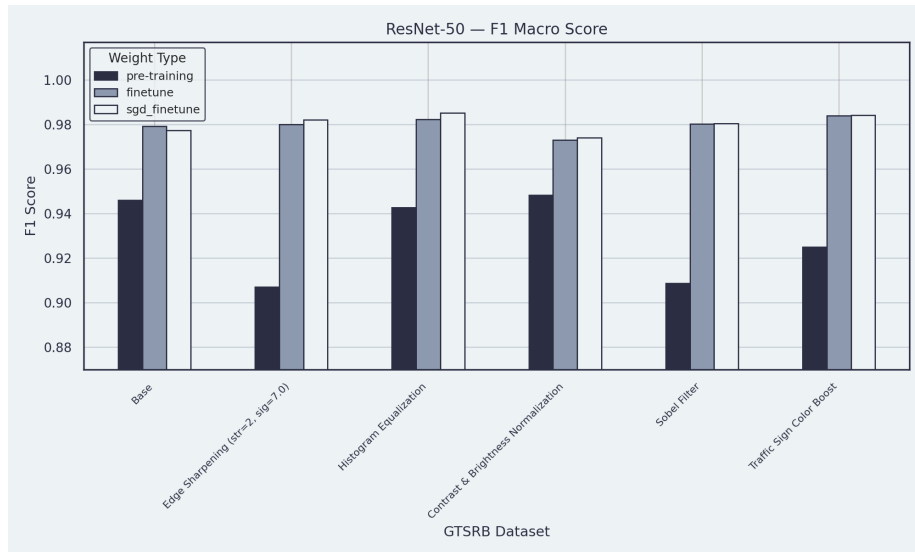


Figure 4: ResNet50 F1 Macro Score

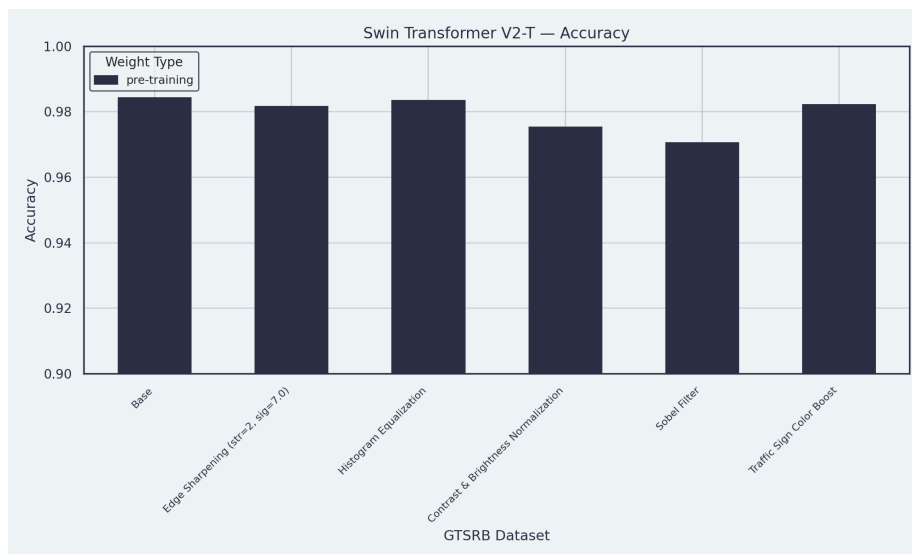


Figure 5: SwinTransformer Accuracy

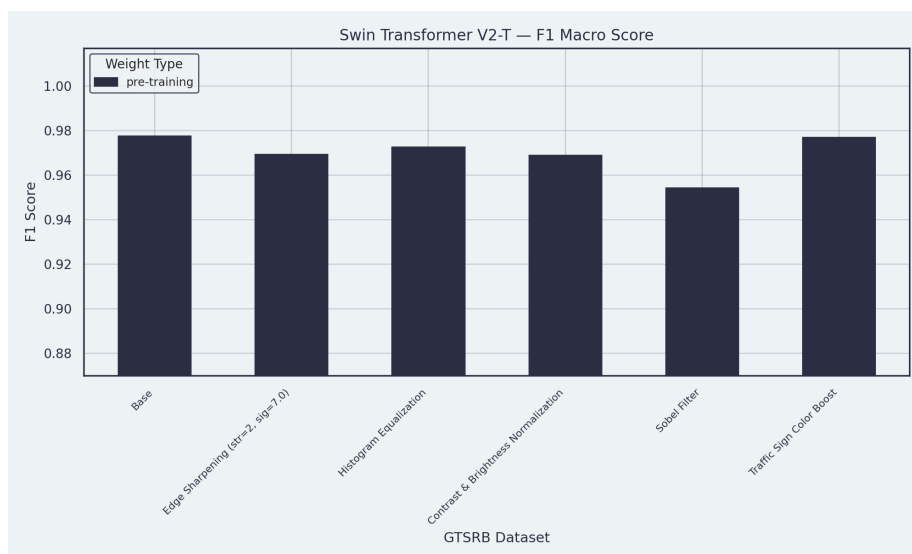


Figure 6: SwinTransformer F1 Macro Score

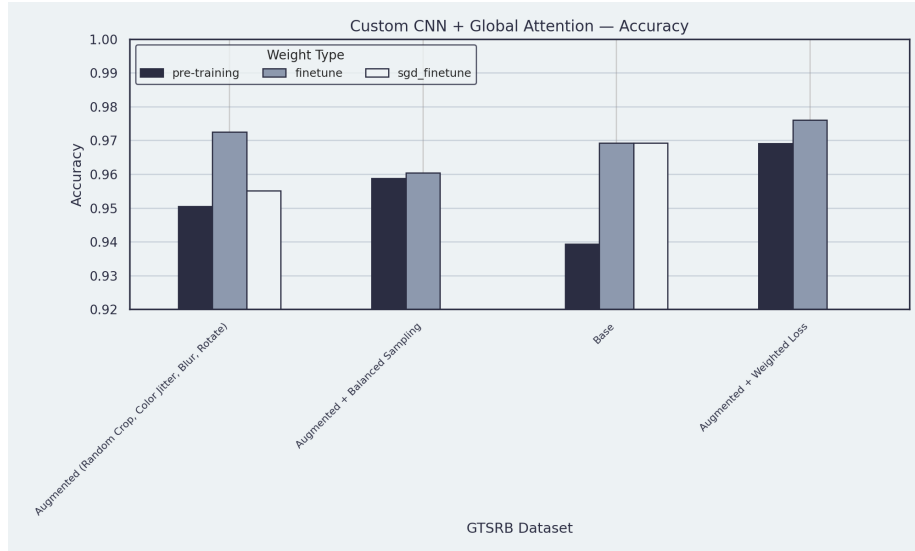


Figure 7: Custom Model Augmentation and Class Imbalance Accuracy

Histogram equalization had an improvement in all models. A likely cause is that it also functions as a kind of normalisation of the the input.

11.3 Augmentation Results

Training with augmentation showed a small improvement compared to the baseline. As it can be seen in 7 and 8 the from augmentation improvement was not huge, probably because GTSRB is already a large dataset with many different lighting conditions and image variations.

Still, light augmentation helped create more varied training samples and improved robustness.

11.4 Class Imbalance Results

After discovering the class imbalance, weighted loss and balanced sampling were tested.

While the weighted loss improved the models performance the balanced sampling did not, likely since the validation set has the same class distribution as the training set nudging performance to overrepresented classes.

This showed that checking class balance is important before and during training.

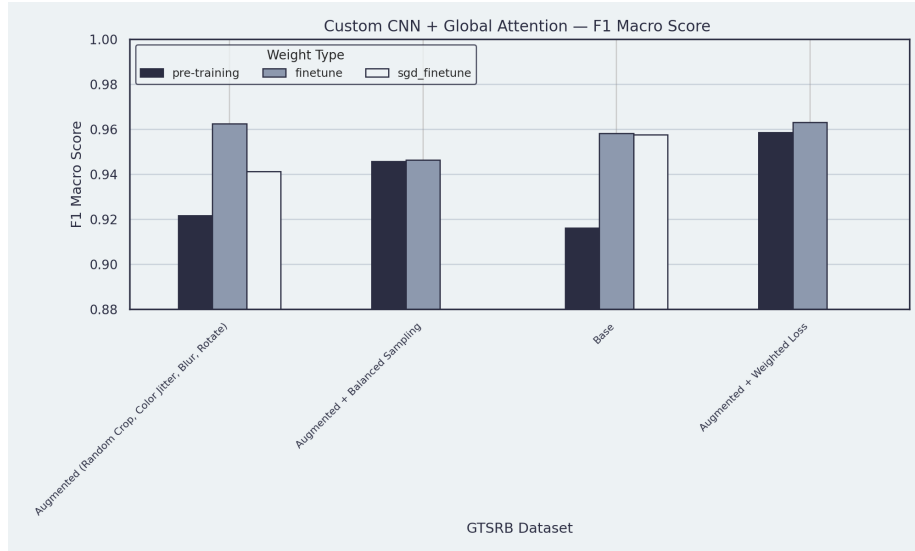


Figure 8: Custom Model Augmentation and Class Imbalance F1 Macro Score

12 Final Best Model Experiments

The information from all previous experiments was used to create the best possible model setup. Since ResNet-50 was the best general model, it was selected for the final experiments.

Two final ResNet-50 setups were trained.

12.1 ResNet-50 with Weighted Loss

This setup used:

- ResNet-50
- histogram equalization
- augmentations
- weighted loss
- base training
- SGD fine-tuning

12.2 ResNet-50 with Balanced Sampling

This setup used:

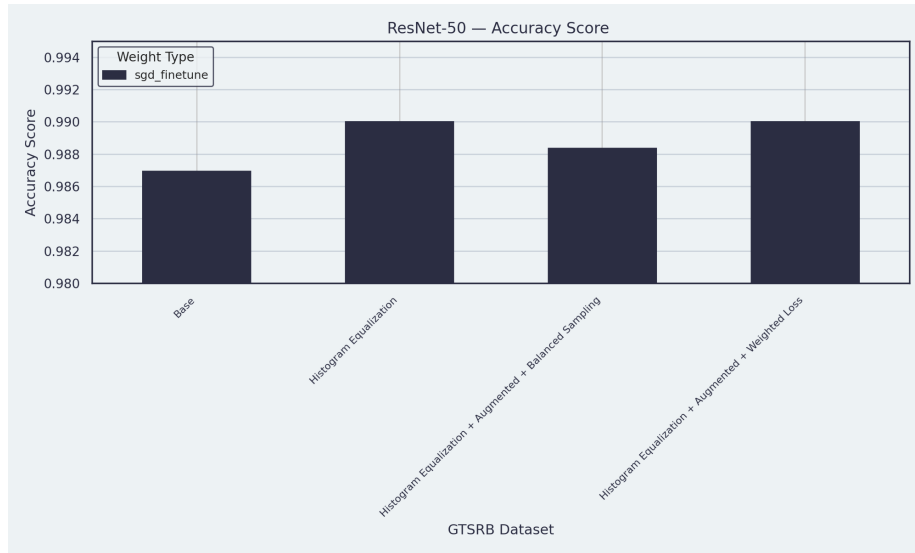


Figure 9: ResNet-50 best model Accuracy

- ResNet-50
- histogram equalization
- augmentations
- balanced sampling
- base training
- SGD fine-tuning

As seen in Figure 9 and 10, adding all performance increasing optimizations together did not bring the hoped-for combined performance increase. Adding augmentation and weighted loss to histogram equalization did not provide any performance boost on the validation set, while balanced sampling even made the performance both on accuracy and F1 score worse. Overall reaching a accuracy of 99% is still a very good score for this dataset.

13 Discussion

The experiments showed that image processing does not automatically improve model performance.

Some processing strategies can make images look cleaner or more structured for humans, but they may remove important information for the model. This

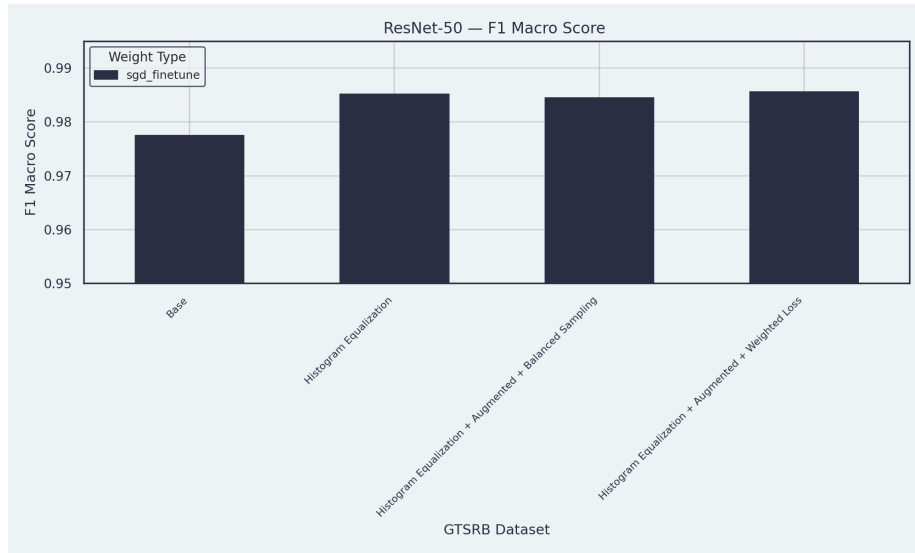


Figure 10: ResNet-50 best model F1 Macro Score

was especially visible with Sobel filtering, which likely removed too much fine information from the image.

The best image processing method was histogram equalization. This probably helped because it made contrast and lighting more consistent across images. However, it also adds extra preprocessing time during inference. In an online or real-time system, this additional inference cost could become a disadvantage.

Augmentation gave a small improvement. This makes sense because augmentation creates more variation during training. However, because the GT-SRB dataset is already quite large and contains different lighting conditions, the improvement was not as strong as it might be on a smaller dataset.

The class imbalance experiments were very useful. Weighted loss and balanced sampling both improved performance, showing that class distribution should always be checked before training.

The custom model was also an important part of the project. Even though it did not outperform ResNet-50 or Swin Transformer-T, it achieved competitive performance while being smaller and faster. This shows that custom models can be useful when designed around the requirements of the task.

14 What I Learned

This project gave several important insights.

First, image processing with sophisticated deep learning models can make performance worse if it creates unnatural images or removes important infor-

mation. Most of the time, preprocessing should be light and should not destroy useful image details.

Second, histogram equalization was useful because it acted like a form of normalization for image contrast. However, it also increases inference time because every image has to be preprocessed.

Third, class balance is very important. The class imbalance in the dataset affected training, and methods like weighted loss and balanced sampling helped improve performance.

Fourth, light augmentation can improve performance by creating more varied training samples. This is especially useful for underrepresented classes.

Finally, creating a custom model can still be valuable. My custom hybrid model was smaller and faster than ResNet-50 and Swin Transformer-T, while still reaching competitive accuracy. This showed that a model designed for the specific problem can be useful even if it does not reach the absolute highest score.

15 Conclusion

This project tested different image processing strategies, augmentations, model architectures, and class imbalance methods on the GTSRB Traffic Sign dataset.

The experiments showed that strong image processing does not always help. In many cases, it made performance worse because useful image information was removed. Histogram equalization was the only image processing method that improved performance clearly.

Augmentation gave a small improvement, while class imbalance handling with weighted loss and balanced sampling gave useful performance gains.

The best final approach was based on ResNet-50 using histogram equalization, augmentation, class imbalance handling, and SGD fine-tuning. This setup reached slightly above 99% accuracy and 98.5% F1-score.

Overall, the project gave useful insight into how preprocessing, augmentation, optimizer choice, and class imbalance handling can affect traffic sign classification performance.

16 References

- GTSRB dataset information: <https://paperswithcode.com/dataset/gtsrb>
- Vision Studio repository: https://github.com/mozi30/CV_VisionStudio